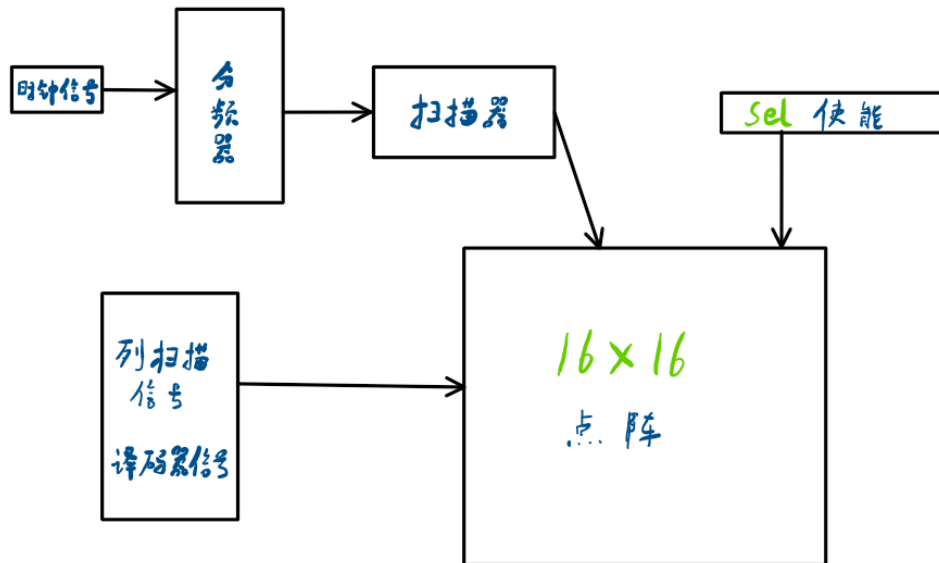


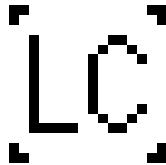
数字逻辑实验报告

实验 6

学 期	2024-2025 学年第 1 学期	实验日期	2024/12/3	
学 院	计算机学院	组 号	学 号	姓 名
专 业	计算机科学与技术（实验班）	15	23071005	侯景祺
班 级	230710			
评 阅 内 容				
任务一	任务二	总结		成 绩
题 目	实验 6: <u>复杂接口设计</u>			
一、实验目的 通过本实验内容的学习，掌握常用复杂接口的工作原理及控制方法，对时序的控制更加熟练。 通过 LED 点阵、SPI 接口、I2C 接口等复杂接口电路的设计，掌握常用复杂接口的工作原理及控制方法，学会使用 VerilogHDL 设计复杂接口的控制器。 二、任务一设计与实现 1. 要求 (1)系统时钟为 50MHz，根据需要设计分频器。 (2)自定义显示内容，点亮 LED。 2. 设计思路 初步设计在 LED 点阵屏上显示“LC”字样。可以使用类似之前实验点亮 7 段 LED 灯的扫描思路，并且给程序输入希望点亮的 LED 灯。试验台的两个 3-8 译码器可以将 4 个引脚拓展为 16 位信号，从而减少编程时所需定义的列。				



其中如下为我希望显示出的 16×16 图像（带边角花纹的“LC”字样）



3. 详细设计

我使用了分频器、扫描器、顶层电路三个电路。其中如下为核心部件扫描器，它定义了要画出的图形。图中记录绘图的代码已进行标注。

```

1 module scanner(clk, I_ROW, I_COL);
2     input clk;
3
4     output reg [15:0] I_ROW = 16'b0;
5     output reg [3:0] I_COL = 4'b0;
6
7     always@(posedge clk)
8         if (I_COL < 16)
9             I_COL <= I_COL + 1;
10        else
11            I_COL <= 4'b0;
12
13    always@(I_COL)
14        case(I_COL)
15            0: I_ROW = 16'b1100_0000_0000_0011; //corner
16            1: I_ROW = 16'b1000_0000_0000_0001; //corner
17            2: I_ROW = 16'b0001_1111_1111_1000; //L
18            3: I_ROW = 16'b0000_0000_0000_1000; //L
19            4: I_ROW = 16'b0000_0000_0000_1000; //L
20            5: I_ROW = 16'b0000_0000_0000_1000; //L
21            6: I_ROW = 16'b0000_0000_0000_1000; //L
22            7: I_ROW = 16'b0000_0000_0000_0000; //BLANK
23            8: I_ROW = 16'b0000_0111_1110_0000; //C
24            9: I_ROW = 16'b0000_1000_0001_0000; //C
25            10: I_ROW = 16'b0001_0000_0000_1000; //C
26            11: I_ROW = 16'b0001_0000_0000_1000; //C
27            12: I_ROW = 16'b0000_1000_0001_0000; //C
28            13: I_ROW = 16'b0000_0100_0010_0000; //C
29            14: I_ROW = 16'b1000_0000_0000_0001; //corner
30            15: I_ROW = 16'b1100_0000_0000_0011; //corner
31            default: I_ROW = 16'b0;
32        endcase
33    endmodule
    
```

以下为顶层文件。

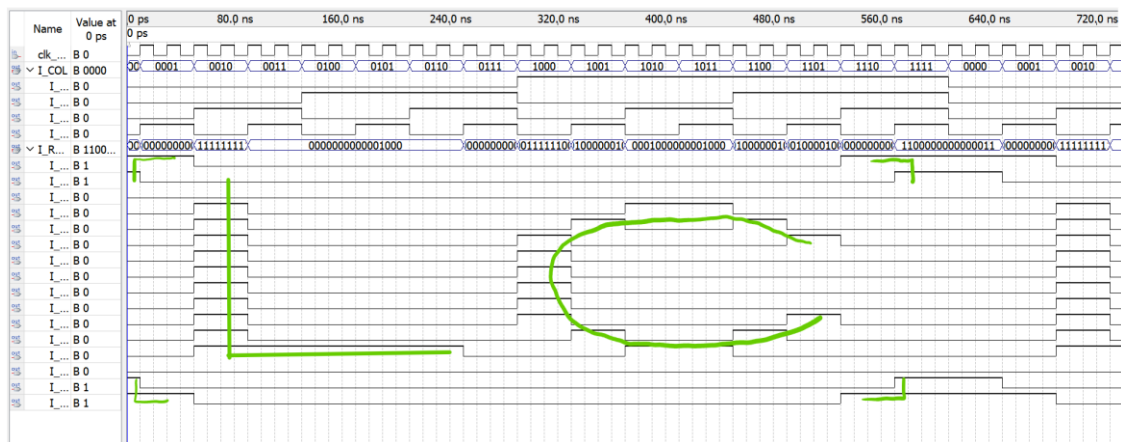
```

1 module top(clk_50mhz, I_COL, I_ROW);
2     input clk_50mhz;
3     wire clk;
4
5     output [15:0] I_ROW;
6     output [3:0] I_COL;
7
8     frequency_divider(.clk_50mhz(clk_50mhz),.clk(clk));
9     scanner(.clk(clk), .I_COL(I_COL), .I_ROW(I_ROW));
10
11 endmodule

```

4. 仿真验证

如下图，为电路的仿真结果。它呈现出 LC 样图形，与希望画出的图形一致。



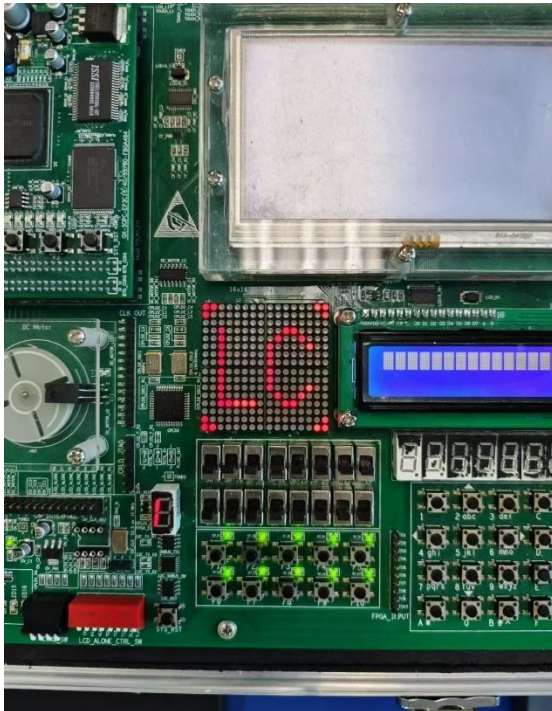
5. 引脚分配

如下图，I_COL 与 I_ROW 被分配 16×16 灯的引脚；clk 被分配硬件中的时钟信号。

Node Name	Direction	Location	I/O Bank
I_COL[3]	Output	PIN_C4	8
I_COL[2]	Output	PIN_A16	7
I_COL[1]	Output	PIN_A15	7
I_COL[0]	Output	PIN_A14	7
I_ROW[15]	Output	PIN_A4	8
I_ROW[14]	Output	PIN_A5	8
I_ROW[13]	Output	PIN_A6	8
I_ROW[12]	Output	PIN_B6	8
I_ROW[11]	Output	PIN_E11	7
I_ROW[10]	Output	PIN_C13	7
I_ROW[9]	Output	PIN_F11	7
I_ROW[8]	Output	PIN_C15	7
I_ROW[7]	Output	PIN_E14	7
I_ROW[6]	Output	PIN_B7	8
I_ROW[5]	Output	PIN_B8	8
I_ROW[4]	Output	PIN_B9	8
I_ROW[3]	Output	PIN_B10	8
I_ROW[2]	Output	PIN_D10	8
I_ROW[1]	Output	PIN_F9	8
I_ROW[0]	Output	PIN_A13	7
clk_50mhz	Input	PIN_T1	2

6. 实验现象

16×16 面板上面能够显示 LC 字样。如下图所示。



三、拓展任务设计与实现

1. 要求

在实验一的基础上，再添加一些图形，并且使得这些图形能以分屏、滚动的方式呈现在 LED 点阵上面。

2. 设计思路

我又画出了一个 16×16 的图像（如下），并且将其像素点明暗信息记录在程序中，使用数组存储 32 个 16 位明暗信息。使用循环可以依次展示这些图像信息。



3. 详细设计

如下图，我使用数组存储两个图像的信息，在下面循环部分对数组内容进行调用，有效简化了循环的逻辑。

```

1  module scanner(clk, I_ROW, I_COL, clk_lhz);
2      input clk, clk_lhz;
3      output reg [15:0] I_ROW = 16'b0;
4      output reg [3:0] I_COL = 4'b0;
5      reg [10:0] flag;
6      reg [15:0] graph [31:0];
7      initial
8      begin
9          graph [0]=16'b1100_0000_0000_0011;//corner
10         graph [1]=16'b1000_0000_0000_0001;//corner
11         graph [2]=16'b0001_1111_1111_1000;//L
12         graph [3]=16'b0000_0000_0000_1000;//L
13         graph [4]=16'b0000_0000_0000_1000;//L
14         graph [5]=16'b0000_0000_0000_1000;//L
15         graph [6]=16'b0000_0000_0000_1000;//L
16         graph [7]=16'b0000_0000_0000_0000;//BLANK
17         graph [8]=16'b0000_0111_1110_0000;//C
18         graph [9]=16'b0000_1000_0001_0000;//C
19         graph [10]=16'b0001_0000_0000_1000;//C
20         graph [11]=16'b0001_0000_0000_1000;//C
21         graph [12]=16'b0000_1000_0001_0000;//C
22         graph [13]=16'b0000_0100_0010_0000;//C
23         graph [14]=16'b0000_0000_0000_0000;//BLANK
24         graph [15]=16'b0000_0000_0000_0000;//BLANK
25         graph [16]=16'b0000_1110_0010_0000;//S
26         graph [17]=16'b0001_0001_0001_0000;//S
27         graph [18]=16'b0000_1000_1111_0000;//S
28         graph [19]=16'b0000_0001_0000_0000;//t
29         graph [20]=16'b0000_0011_1110_0000;//t
30         graph [21]=16'b0000_0001_0001_0000;//t
31         graph [22]=16'b0000_0000_1110_0000;//a
32         graph [23]=16'b0000_0001_0001_0000;//a
33         graph [24]=16'b0000_0000_1110_0000;//a
34         graph [25]=16'b0000_0000_0001_0000;//a
35         graph [26]=16'b0000_0001_0000_0000;//t
36         graph [27]=16'b0000_0011_1110_0000;//t
37         graph [28]=16'b0000_0001_0001_0000;//t
38         graph [29]=16'b0000_0001_1110_0000;//e
39         graph [30]=16'b1000_0010_1001_0001;//corner&e
40         graph [31]=16'b1100_0001_1010_0011;//corner&e
41     end
42
43     always@(posedge clk)
44     if (I_COL < 16)
45         I_COL <= I_COL + 1;
46     else
47         I_COL <= 4'b0;
48
49     always@(posedge clk_lhz)
50     if (flag < 18)
51         flag <= flag + 1;
52     else
53         flag <= 0;
54
55
56     always@(I_COL, flag)
57     if (flag>2)
58     begin
59         I_ROW = graph[I_COL+flag-2];
60     end
61     else begin
62         case(flag)
63             0:
64                 I_ROW = graph[I_COL];
65             1:
66                 I_ROW = graph[I_COL+16];
67             2:
68                 I_ROW = graph[I_COL];
69         endcase
70     end
71 endmodule

```

如下为顶层文件，此处不多赘述。

```

1  module expand_task(clk_50mhz, I_COL, I_ROW);
2      input clk_50mhz;
3      wire clk, clk_lhz;
4
5      output [15:0] I_ROW;
6      output [3:0] I_COL;
7
8      frequency_divider(.clk_50mhz(clk_50mhz), .clk(clk), .clk_lhz(clk_lhz));
9      scanner(.clk(clk), .I_COL(I_COL), .I_ROW(I_ROW), .clk_lhz(clk_lhz));
10
11 endmodule

```

4. 仿真验证

如下图，一开始程序进行了分屏（即完整地、不移动地显示图像，先显示 LC，再显示 State 字样）随后，图像重置为 LC 字样，并且每过相同时间（时钟刻），都会向左移动一个像素。经过 16 个时钟刻后，图像已由 LC 完全移动为 State 字样。

下图中受限于空间，无法完全展示此过程。但是可以看出 LC 字样已经逐渐淡出，而 S 字样逐渐出现、向左移动。



5. 引脚分配

与实验一相同，不多赘述。

Node Name	Direction	Location	I/O Bank
 I_COL[3]	Output	PIN_C4	8
 I_COL[2]	Output	PIN_A16	7
 I_COL[1]	Output	PIN_A15	7
 I_COL[0]	Output	PIN_A14	7
 I_ROW[15]	Output	PIN_A4	8
 I_ROW[14]	Output	PIN_A5	8
 I_ROW[13]	Output	PIN_A6	8
 I_ROW[12]	Output	PIN_B6	8
 I_ROW[11]	Output	PIN_E11	7
 I_ROW[10]	Output	PIN_C13	7
 I_ROW[9]	Output	PIN_F11	7
 I_ROW[8]	Output	PIN_C15	7
 I_ROW[7]	Output	PIN_E14	7
 I_ROW[6]	Output	PIN_B7	8
 I_ROW[5]	Output	PIN_B8	8
 I_ROW[4]	Output	PIN_B9	8
 I_ROW[3]	Output	PIN_B10	8
 I_ROW[2]	Output	PIN_D10	8
 I_ROW[1]	Output	PIN_F9	8
 I_ROW[0]	Output	PIN_A13	7
 clk_50mhz	Input	PIN_T1	2

6. 实验现象

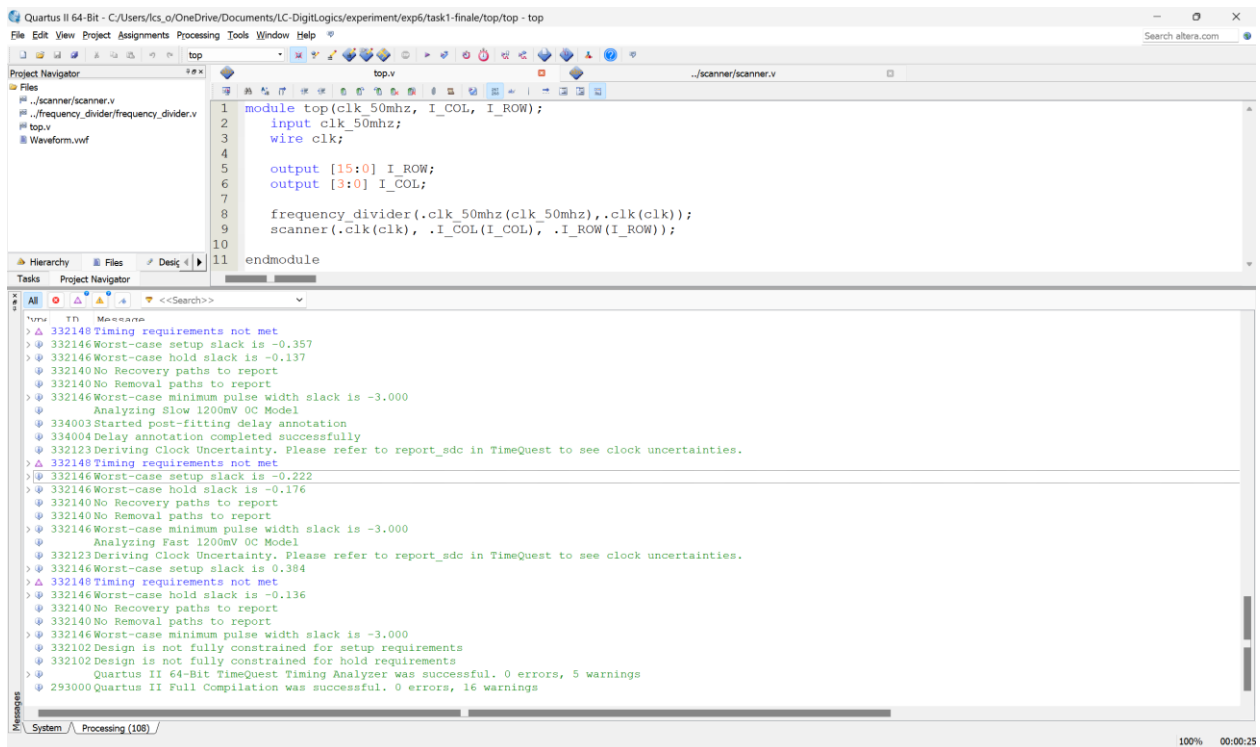
首先，LED 点阵先间隔一个时钟刻显示 LC、State 字样，随后回到 LC 字样，字开始向左滚动，间隔为 1 个时钟刻。整个过程完成需要 18 个时钟刻（2 个为分屏显示，16 个为滚动显示）。

四、总结

实验一中，由于对屏幕不熟悉，导致字幕显示反转、镜像，后来通过修改引脚以及扫描电路化解了问题。

在实验二中，由于不熟悉语法，未能将数组正确地在 initial 里面定义，从而导致报错。

附图 1：任务一顶层及综合通过图



附图 2：拓展任务顶层及综合通过图

